

Un modello della programmazione concorrente

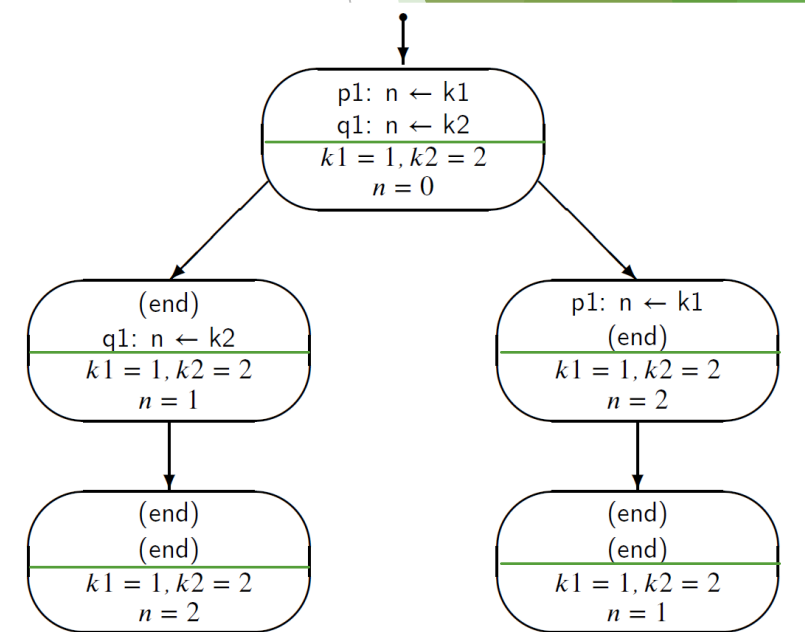
Scopo del modello

L'insieme dei **comportamenti possibili** di un programma concorrente può essere descritto come un **sistema di transizioni**

► **Interleaving** e **scelte non deterministiche**

Il sistema di transizioni è definito dalla **semantica** del linguaggio di programmazione

Usando un **modello** potremmo **analizzare le problematiche di sincronizzazione** sul sistema di transizioni di un linguaggio semplice...



La base: un linguaggio imperativo minimale

Sintassi:

$$e ::= n \mid e + e \mid !\ell \mid \ell := e \mid \text{skip} \mid e; e$$

Tipi: int, unit

In sostanza:

- ▶ somme di interi, lettura (!) e scrittura (:=) di locazioni di memoria ℓ , skip e sequenze (;)
- ▶ assegnamenti come espressioni di tipo unit con side effect (come in OCaml)

La base: un linguaggio imperativo minimale

Regole di tipo

Γ mi dice solo se la locazione esiste

$$\Gamma \vdash n : \text{int} \quad \frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}} \quad \frac{\Gamma(\ell) = \text{intloc}}{\Gamma \vdash !\ell : \text{int}}$$

$$\frac{\Gamma(\ell) = \text{intloc} \quad \Gamma \vdash e : \text{int}}{\Gamma \vdash \ell := e : \text{unit}} \quad \frac{\Gamma \vdash e_1 : \text{unit} \quad \Gamma \vdash e_2 : \text{unit}}{\Gamma \vdash e_1 ; e_2 : \text{unit}}$$

$$\Gamma \vdash \text{skip} : \text{unit}$$

Semantica (small-step)

(op+) $\langle n_1 + n_2, s \rangle \rightarrow \langle n, s \rangle$ se $n = n_1 + n_2$

Le configurazioni sono coppie $\langle e, s \rangle$ dove e è un'espressione ed s è uno stato che associa locazioni a interi

(comp1)
$$\frac{\langle e_1, s \rangle \rightarrow \langle e'_1, s' \rangle}{\langle e_1 + e_2, s \rangle \rightarrow \langle e'_1 + e_2, s' \rangle}$$

(comp2)
$$\frac{\langle e_2, s \rangle \rightarrow \langle e'_2, s' \rangle}{\langle n + e_2, s \rangle \rightarrow \langle n + e'_2, s' \rangle}$$

(deref) $\langle !\ell, s \rangle \rightarrow \langle n, s \rangle$ se $\ell \in \text{dom}(s)$ e $s(\ell) = n$

Aggiornamento della memoria

(assign1) $\langle \ell := n, s \rangle \rightarrow \langle \text{skip}, s + \{\ell \mapsto n\} \rangle$ se $\ell \in \text{dom}(s)$

(assign2)
$$\frac{\langle e, s \rangle \rightarrow \langle e', s' \rangle}{\langle \ell := e, s \rangle \rightarrow \langle \ell := e', s' \rangle}$$

In generale, la valutazione di un'espressione può portare a modificare lo stato

(seq1) $\langle \text{skip}; e_2, s \rangle \rightarrow \langle e_2, s \rangle$

(seq2)
$$\frac{\langle e_1, s \rangle \rightarrow \langle e'_1, s' \rangle}{\langle e_1; e_2, s \rangle \rightarrow \langle e'_1; e_2, s' \rangle}$$

Esempio

Eseguiamo il programma

$$e = \ell_1 := 3; \ell_2 := !\ell_1 + 1$$

nella memoria

$$s = \{\ell_1 \mapsto 0; \ell_2 \mapsto 0\}$$

Risultato:

$$\begin{aligned} &\langle \ell_1 := 3; \ell_2 := !\ell_1 + 1, \{\ell_1 \mapsto 0; \ell_2 \mapsto 0\} \rangle \rightarrow \\ &\quad \langle \text{skip}; \ell_2 := !\ell_1 + 1, \{\ell_1 \mapsto 3; \ell_2 \mapsto 0\} \rangle \rightarrow \\ &\quad \langle \ell_2 := !\ell_1 + 1, \{\ell_1 \mapsto 3; \ell_2 \mapsto 0\} \rangle \rightarrow \\ &\quad \langle \ell_2 := 3 + 1, \{\ell_1 \mapsto 3; \ell_2 \mapsto 0\} \rangle \rightarrow \\ &\quad \langle \ell_2 := 4, \{\ell_1 \mapsto 3; \ell_2 \mapsto 0\} \rangle \rightarrow \\ &\quad \langle \text{skip}, \{\ell_1 \mapsto 3; \ell_2 \mapsto 4\} \rangle \end{aligned}$$

Estensione concorrente

Scelte di progettazione per l'estensione concorrente thread

- ▶ con memoria condivisa (**shared memory**)
- ▶ eseguiti in modo concorrente con **interleaving**
- ▶ privi di identità
- ▶ la cui terminazione non è osservabile dal resto del programma
- ▶ che non possono essere forzati a terminare dall'esterno
- ▶ che non restituiscono un risultato al termine

Estensione concorrente

Modelleremo i thread come **espressioni** composte tramite un operatore di **composizione parallela**: $e_1 | e_2$

- e_1 ed e_2 sono i due thread in esecuzione concorrente

Ambiguità terminologiche...

- l'operatore $|$ si chiama tradizionalmente "**parallel** composition, ma si interpreta come **composizione concorrente**...
- anche gli operandi di $|$ sono tradizionalmente chiamati "**processi**" anche se in questo caso il loro comportamento sarà più simile a quello dei **thread**

Estensione concorrente

Sintassi estesa:

$e ::= n \mid e + e \mid !\ell \mid \ell := e \mid skip \mid e; e \mid e|e$

Tipi estesi: int, unit, proc

Regole di tipo aggiunte:

$$\frac{\Gamma \vdash e: \text{unit}}{\Gamma \vdash e: \text{proc}}$$

$$\frac{\Gamma \vdash e_1: \text{proc} \quad \Gamma \vdash e_2: \text{proc}}{\Gamma \vdash e_1 \mid e_2: \text{proc}}$$

Estensione concorrente

Regole semantiche aggiunte:

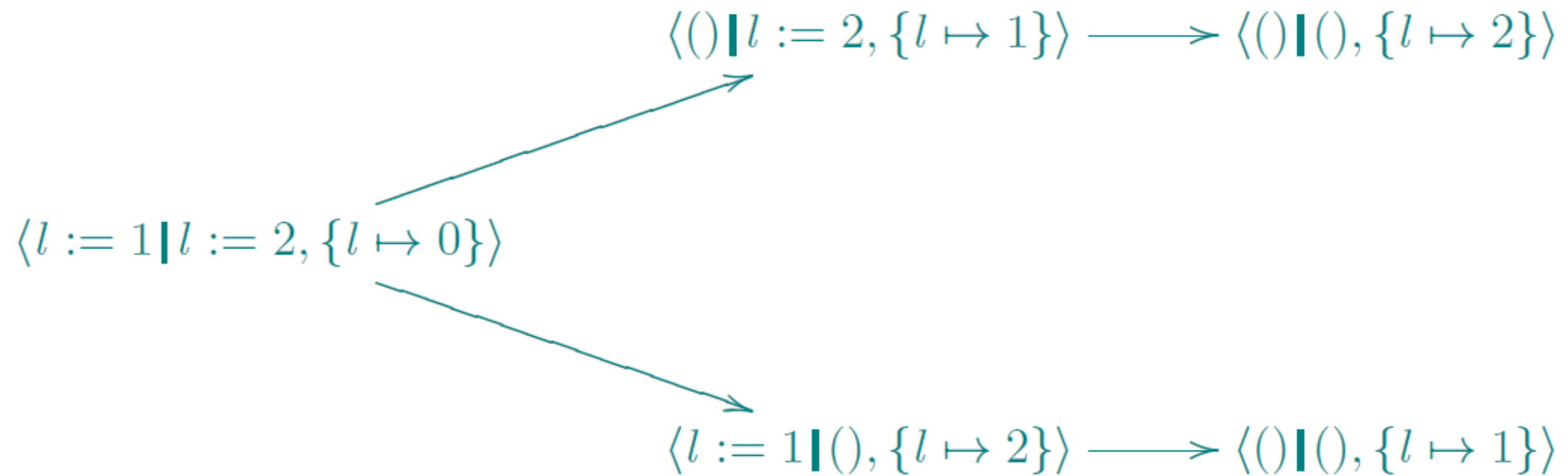
$$\text{(parallel1)} \quad \frac{\langle e_1, s \rangle \rightarrow \langle e'_1, s' \rangle}{\langle e_1 | e_2, s \rangle \rightarrow \langle e'_1 | e_2, s' \rangle}$$

$$\text{(parallel2)} \quad \frac{\langle e_2, s \rangle \rightarrow \langle e'_2, s' \rangle}{\langle e_1 | e_2, s \rangle \rightarrow \langle e_1 | e'_2, s' \rangle}$$

Queste regole **descrivono in modo semplice** il meccanismo di **interleaving**

- ▶ due (o più) thread concorrenti possono avanzare **uno per volta**
- ▶ la **scelta** tra l'applicazione di (parallel1) e (parallel2) è **non deterministica**
- ▶ la **memoria** è **condivisa**

Esempio concorrente



Nota: $()$ è l'abbreviazione di skip

Operazioni atomiche

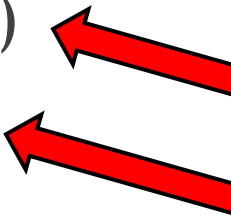
Per come abbiamo definito la semantica, le **operazioni atomiche** del linguaggio (quelle che danno origine a una singola transizione) sono:

- ▶ somma (+)
- ▶ dereferenziazione (!)
- ▶ assegnamento (:=)
- ▶ skip

Operazioni atomiche

Per come abbiamo definito la semantica, le **operazioni atomiche** del linguaggio (quelle che danno origine a una singola transizione) sono:

- ▶ somma (+)
- ▶ dereferenziazione (!)
- ▶ assegnamento (:=)
- ▶ skip



In particolare, **lettura e scrittura in memoria** sono **operazioni atomiche separate** (come a **livello macchina...** vedere ragionamenti precedenti)

Questa semantica cattura bene i comportamenti a basso livello

Esempio concorrente "problematico"

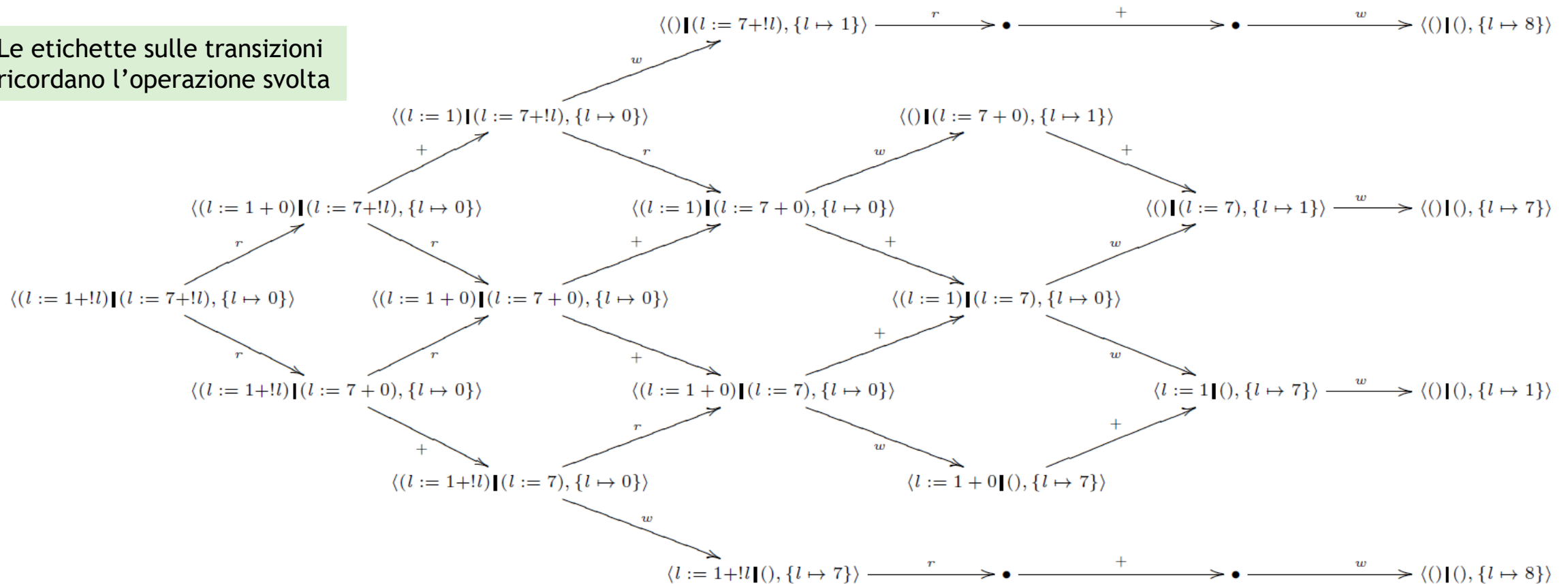
Eseguiamo $(\ell := 1 + !\ell) | (\ell := 7 + !\ell)$ nella memoria $\{\ell \mapsto 0\}$

- ▶ I due thread condividono la locazione ℓ
- ▶ Ognuno cerca di **incrementare** il valore in locazione ℓ
- ▶ Qual è il comportamento di questo programma?

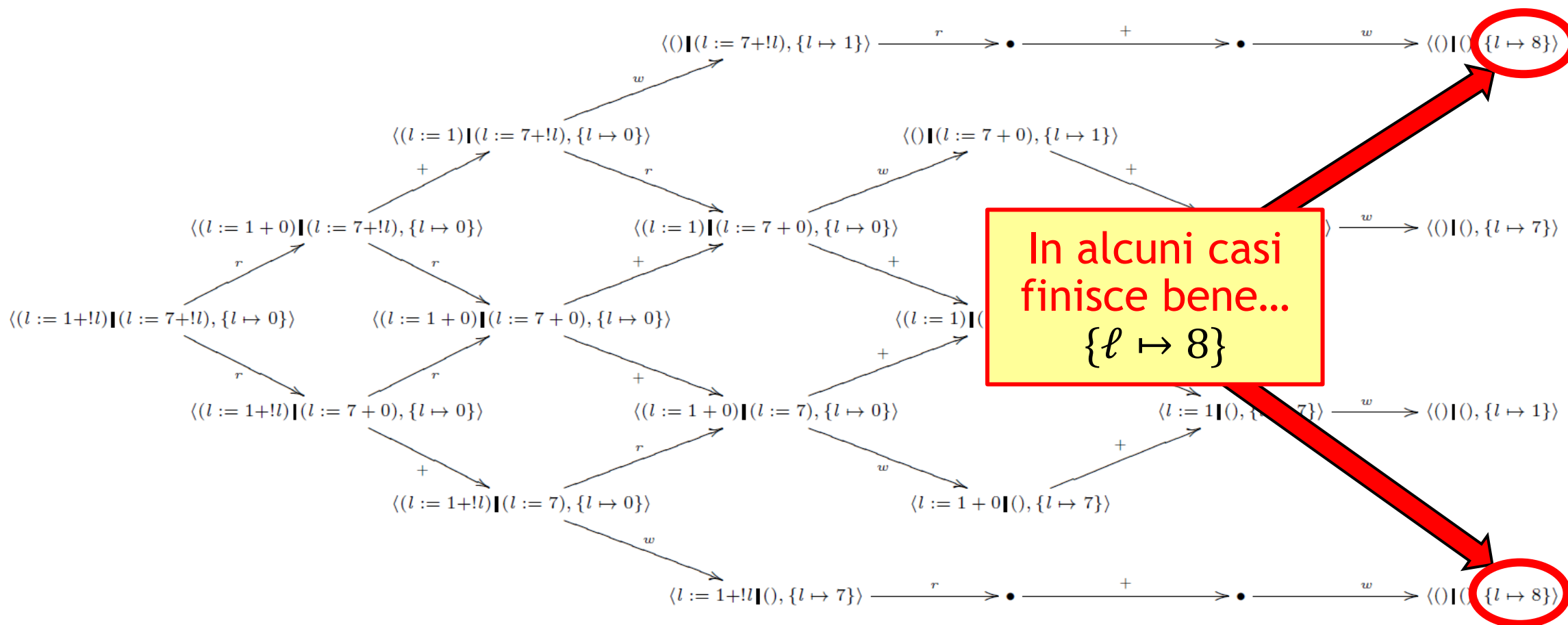
Esempio concorrente "problematico"

La semantica ci fornisce il sistema di transizioni che descrive tutti i comportamenti possibili!

Le etichette sulle transizioni ricordano l'operazione svolta

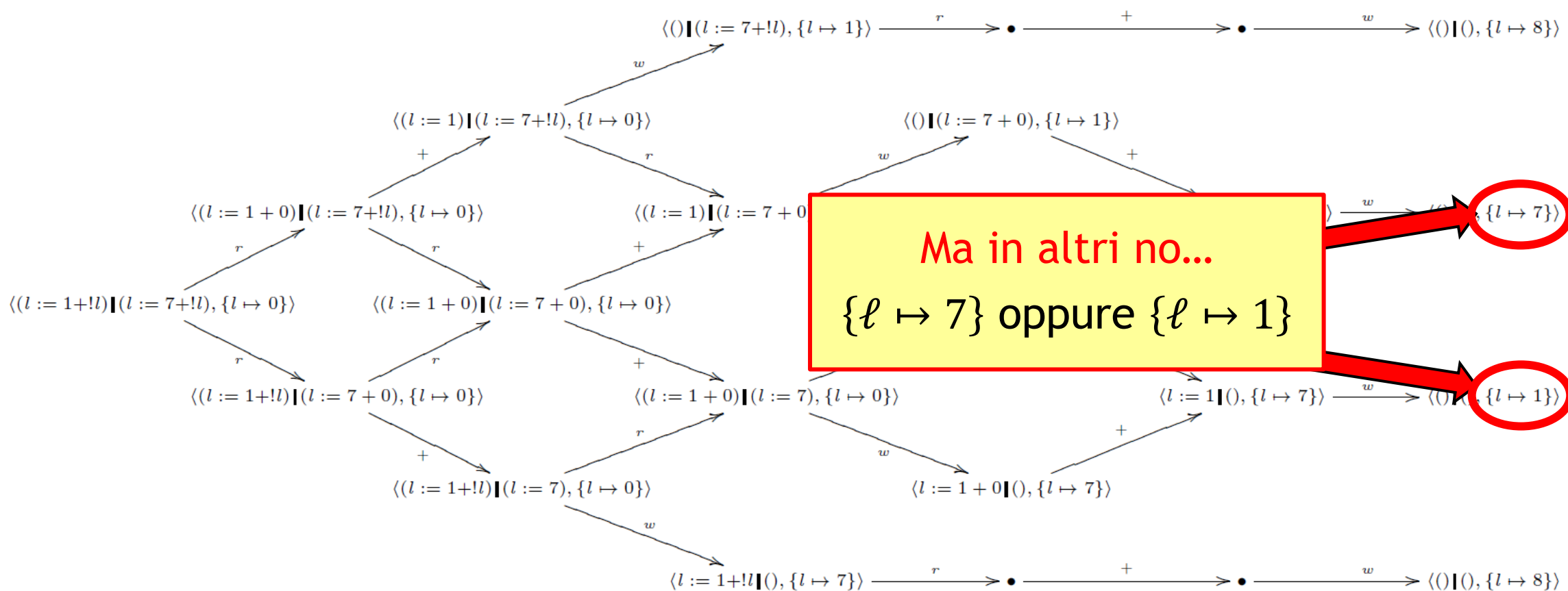


La semantica ci fornisce il sistema di transizioni che descrive tutti i comportamenti possibili!



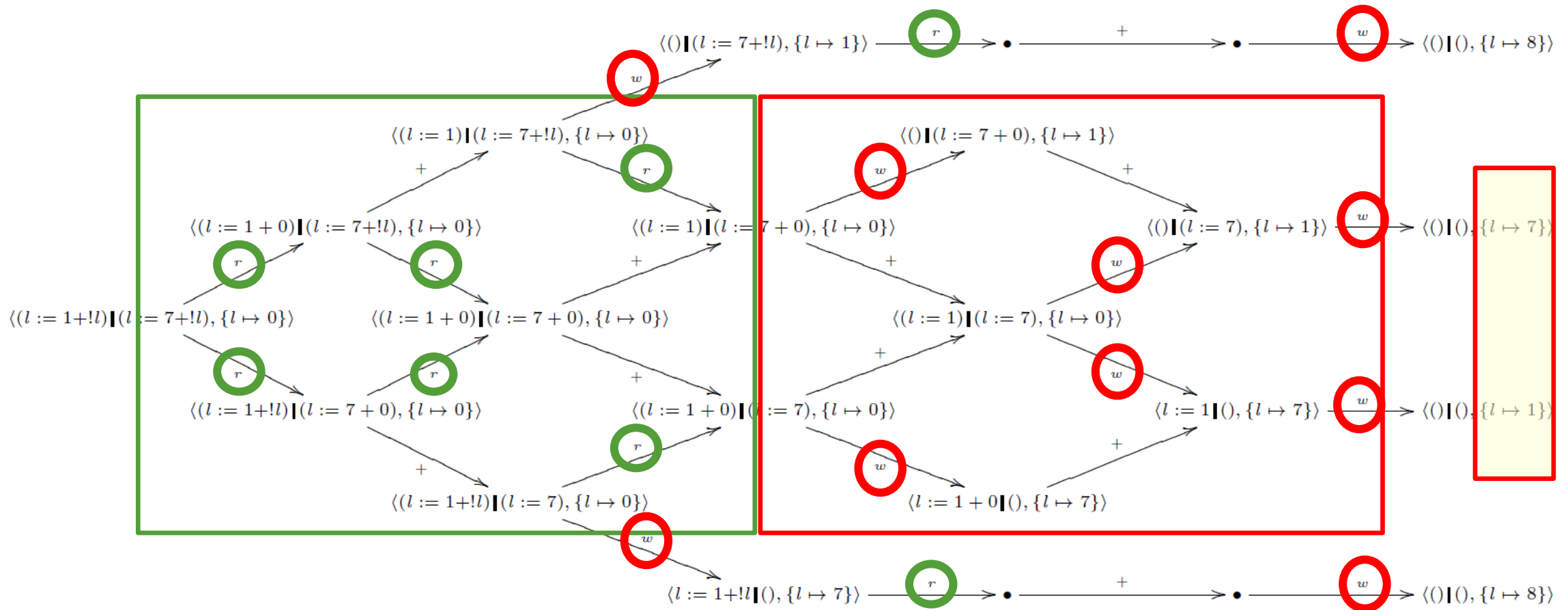
Esempio concorrente "problematico"

La semantica ci fornisce il sistema di transizioni che descrive tutti i comportamenti possibili!



Tutti (e soli) i cammini che portano a risultati sbagliati
fanno **prima** tutte le letture **r** e **poi** tutte le
scritture **w**!

Il sistema di transizioni mostra bene questa proprietà!



Osservazioni

Il **sistema di transizioni** consente di **ragionare** bene sulle proprietà comportamentali dei programmi concorrenti

Ma:

- ▶ C'è un'**esplosione combinatoriale** dello spazio degli stati
- ▶ ossia, a causa dell'interleaving, **il numero degli stati raggiungibili (dimensione del grafo)** può crescere esponenzialmente rispetto alla dimensione del programma

Per gestire questa complessità, è necessario ricorrere a metodi avanzati di **analisi statica**, che permettono di studiare il **comportamento e le proprietà dei programmi concorrenti**, senza dover esplorare ogni stato possibile

Meccanismi di sincronizzazione

Ma quindi come si fa a far funzionare correttamente il programma $(\ell := 1 + !\ell) | (\ell := 7 + !\ell)$?

- ▶ Bisogna introdurre nel linguaggio un **meccanismo di sincronizzazione**, per evitare che un thread legga il valore mentre l'altro lo sta per modificare
- ▶ Rimanendo nel modello, **introduciamo un meccanismo di mutua* esclusione basata su "lock"**, che garantisce che **un solo thread alla volta** possa accedere a una risorsa condivisa
- ▶ Questo meccanismo **astrae i vari meccanismi presenti nei linguaggi di programmazione reali**

* **reciproca**

Un modello della programmazione
concorrente (con i lock)

Primitive di Mutua Esclusione (**mutex**)

Nei linguaggi di programmazione esistono **diverse primitive di sincronizzazione**

Come **esempi**, abbiamo già visto:

- ▶ **sleep** e **wakeup**
- ▶ **send** e **receive**
- ▶ ...

Tutte quante prevedono meccanismi per bloccare e sbloccare processi/thread

Primitive di Mutua Esclusione (**mutex**)

Nel caso di thread concorrenti con shared memory, la **primitiva di sincronizzazione di riferimento** si basa sull'uso di meccanismi di **locking**

- ▶ Servono per assicurare **mutua esclusione** per l'accesso alle locazioni di memoria condivise:
 - ▶ un thread può assicurarsi, per un po' di passi, di essere l'unico ad accedere a una certa locazione di memoria condivisa
- ▶ Le primitive per implementare questo controllo sono **lock** e **unlock**, che consentono rispettivamente di **acquisire** e **rilasciare** il controllo sull'accesso alla memoria condivisa

Primitive di Mutua Esclusione (**mutex**)

Come funziona

A **ogni area di memoria** da condividere e alla quale accedere in mutua esclusione è associato un **mutex** m

- ▶ una entità astratta (un token, un testimone,...)
- ▶ **Acquisizione del mutex**: prima di accedere all'area di memoria protetta, un thread (ad es. T_1) deve acquisire il mutex eseguendo il comando **lock m**
- ▶ **Accesso esclusivo**: se nessun altro thread (ad es. T_2) sta già accedendo a quella memoria, T_1 ottiene il controllo del mutex e può procedere con le sue operazioni
- ▶ **Attesa**: se invece un altro thread (T_2) ha già acquisito il mutex, T_1 viene bloccato fino a quando T_2 non lo rilascia
- ▶ **Rilascio del mutex**: una volta terminato l'accesso alla memoria, T_1 deve rilasciare il mutex eseguendo **unlock m** , consentendo ad altri thread in attesa di procedere

Estendiamo il linguaggio concorrente con le primitive di locking

Sintassi estesa:

$e ::= n \mid e + e \mid !\ell \mid \ell := e \mid skip \mid e; e \mid e|e \mid \textbf{lock } m \mid \textbf{unlock } m$

dove $m \in \mathbb{M}$ (insieme, anche infinito, dei mutex)

Regole di tipo aggiunte: $\Gamma \vdash \textbf{lock } m: \text{unit}$ $\Gamma \vdash \textbf{unlock } m: \text{unit}$

Configurazioni della semantica:

- ▶ invece di $\langle e, s \rangle$ abbiamo $\langle e, s, M \rangle$ con $M: \mathbb{M} \mapsto \{\textbf{true}, \textbf{false}\}$
- ▶ $M(m)$ rappresenta lo stato del mutex m (è **true** se già acquisito)

Estendiamo il linguaggio concorrente con le primitive di locking

Regole semantiche aggiunte:

Solo se il mutex è libero

(lock) $\langle \text{lock } m, s, M \rangle \rightarrow \langle (), s, M + \{m \mapsto \text{true}\} \rangle$ if $\neg M(m)$

(unlock) $\langle \text{unlock } m, s, M \rangle \rightarrow \langle (), s, M + \{m \mapsto \text{false}\} \rangle$

Inoltre, vanno modificate tutte le regole semantiche precedentemente definite, aggiungendo M ovunque... ad esempio:

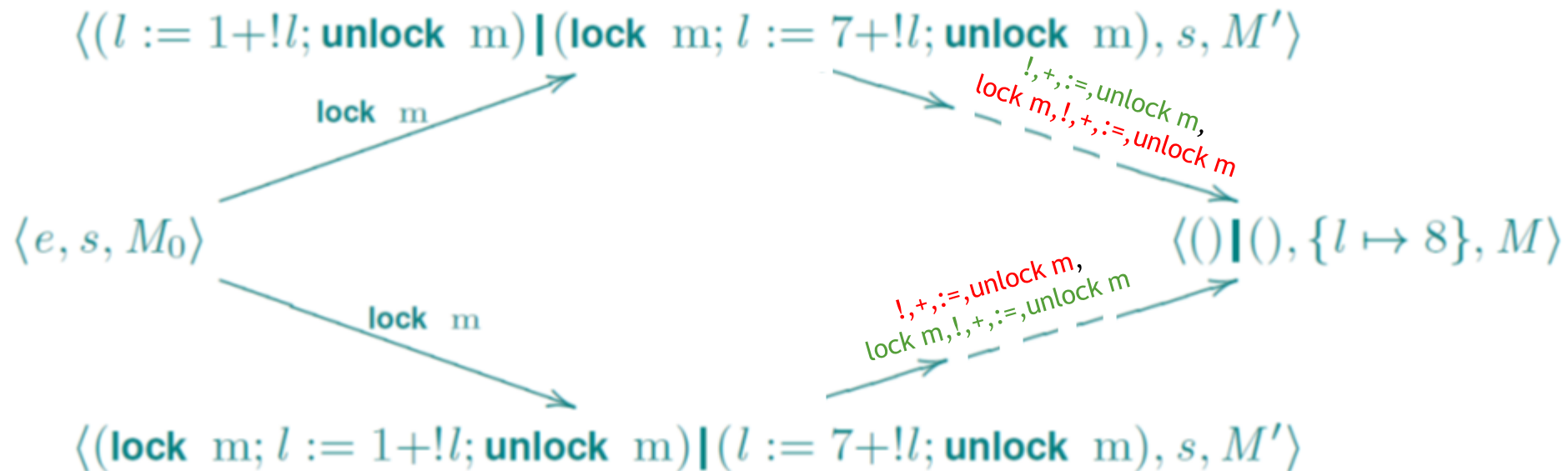
(seq2) $\frac{\langle e_1, s \rangle \rightarrow \langle e'_1, s' \rangle}{\langle e_1; e_2, s \rangle \rightarrow \langle e'_1; e_2, s' \rangle} \quad \longrightarrow \quad \frac{\langle e_1, s, M \rangle \rightarrow \langle e'_1, s', M' \rangle}{\langle e_1; e_2, s, M \rangle \rightarrow \langle e'_1; e_2, s', M' \rangle}$

Usare i mutex

Riprendiamo l'esempio: $(\ell := 1 + !\ell) | (\ell := 7 + !\ell)$ e ora usiamo un **mutex**:

$$e = (\text{lock } m; \ell := 1 + !\ell; \text{unlock } m) | (\text{lock } m; \ell := 7 + !\ell; \text{unlock } m)$$

Il comportamento di e nella memoria $s = \{\ell \mapsto 0\}$ e con stato iniziale dei mutex $M_0 = \{\forall m. (m \mapsto \text{false})\}$ è:



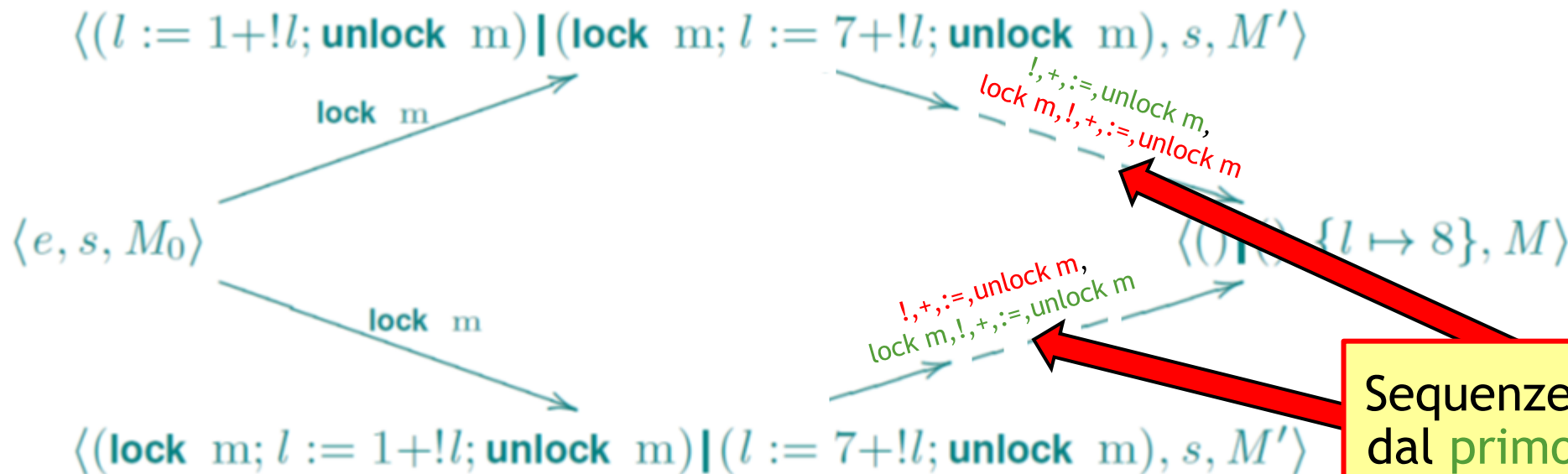
dove: $M' = M_0 + \{m \mapsto \text{true}\}$

Usare i mutex

Riprendiamo l'esempio: $(\ell := 1 + !\ell) | (\ell := 7 + !\ell)$ e ora usiamo un **mutex**:

$$e = (\text{lock } m; \ell := 1 + !\ell; \text{unlock } m) | (\text{lock } m; \ell := 7 + !\ell; \text{unlock } m)$$

Il comportamento di e nella memoria $s = \{\ell \mapsto 0\}$ e con stato iniziale dei mutex $M_0 = \{\forall m. (m \mapsto \text{false})\}$ è:



Sequenze di passi fatti dal **primo thread** e dal **secondo thread**

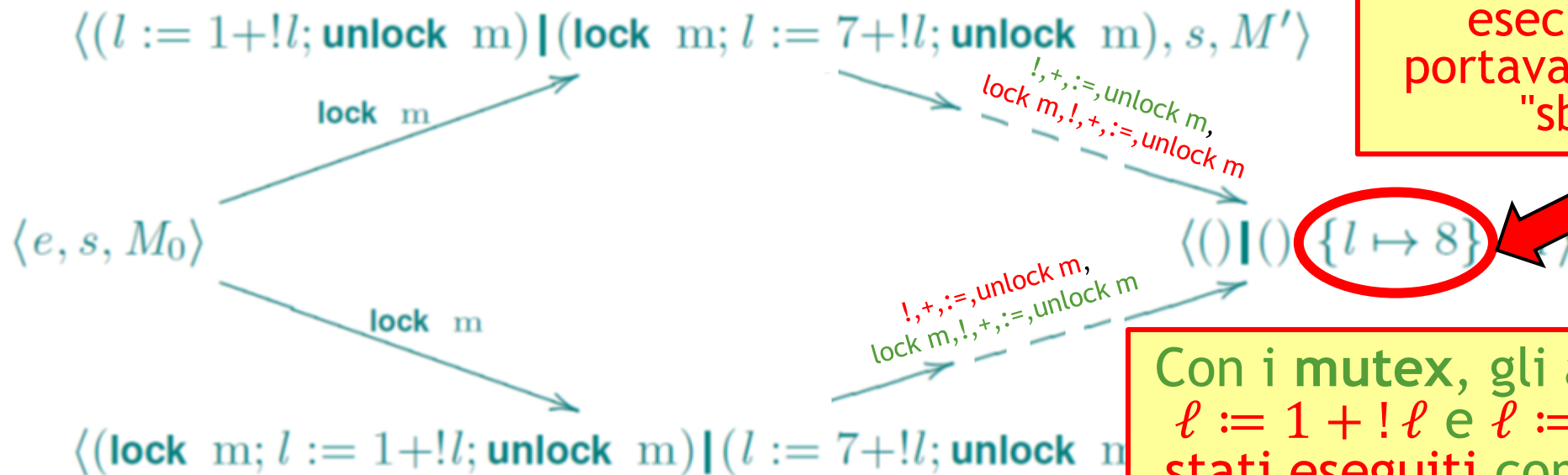
dove: $M' = M_0 + \{m \mapsto \text{true}\}$

Usare i mutex

Riprendiamo l'esempio: $(\ell := 1 + !\ell) | (\ell := 7 + !\ell)$ e ora usiamo un **mutex**:

$$e = (\text{lock } m; \ell := 1 + !\ell; \text{unlock } m) | (\text{lock } m; \ell := 7 + !\ell; \text{unlock } m)$$

Il comportamento di e nella memoria $s = \{\ell \mapsto 0\}$ e con stato iniziale dei mutex $M_0 = \{\forall m. (m \mapsto \text{false})\}$ è:



Sono sparite le
esecuzioni che
portavano a risultati
"sbagliati"

Con i mutex, gli assegnamenti
 $\ell := 1 + !\ell$ e $\ell := 7 + !\ell$ sono
stati eseguiti come se fossero
atomici

dove: $M' = M_0 + \{m \mapsto \text{true}\}$

Usare i mutex

- ▶ Il corretto funzionamento dei mutex si basa sulla capacità della primitiva **lock di bloccare temporaneamente un thread**, evitando accessi concorrenti non controllati
- ▶ Un **uso coerente** dei mutex **in tutti i thread** permette di prevenire interleaving non desiderati e garantisce la sicurezza nell'accesso alla memoria condivisa

Principi per un uso coerente dei mutex:

- ▶ Ogni **locazione** condivisa deve essere **associata a un mutex** dedicato (un mutex può essere associato a più locazioni)
- ▶ **Prima di iniziare** a utilizzare la locazione condivisa, il thread deve eseguire un **lock** sul mutex a essa associato
- ▶ **Dopo aver terminato** di utilizzare la locazione, il thread deve eseguire un **unlock** sul mutex a essa associato

"Coarse-grained" and "fine-grained" locking

L'associazione dei mutex alle locazioni può essere fatta con strategie diverse con implicazioni diverse sulla complessità, l'efficienza e la scalabilità delle applicazioni concorrenti

Esempi ESTREMI:

- **Unico mutex** (o comunque un numero ridotto di mutex) **per** proteggere **tutte le locazioni** (**coarse-grained**):

$$\ell_1, \ell_2, \dots \mapsto m$$

- Un **mutex diverso** (o comunque più mutex per piccole porzioni) **per ogni locazione** (**fine-grained**):

$$\ell_1 \mapsto m_1, \ell_2 \mapsto m_2, \dots$$

"Coarse-grained" and "fine-grained" locking

Vediamo un esempio: thread che accede a due locazioni

► **coarse-grained:**

$$e_{cg} = (\mathbf{lock\ } m; \ell_1 := 1 + !\ell_2; \mathbf{unlock\ } m)$$

► **fine-grained:**

$$e_{fg} = (\mathbf{lock\ } m_1; \mathbf{lock\ } m_2; \ell_1 := 1 + !\ell_2; \mathbf{unlock\ } m_1; \mathbf{unlock\ } m_2)$$

La strategia **coarse-grained**

- richiede **meno lavoro al singolo thread** (una singola operazione di lock)
- ma **riduce la concorrenza** (**lock** m blocca tutti gli altri thread, anche quelli che accedono a locazioni di memoria diverse)

"Coarse-grained" and "fine-grained" locking

Un altro esempio (idealmente, il programma: $x++;y++ \mid x++;z++$):

► coarse-grained:

$$e'_{cg} = (\text{lock } m; \ell_1 := 1 + !\ell_1; \ell_2 := 1 + !\ell_2; \text{unlock } m) \\ \mid (\text{lock } m; \ell_1 := 1 + !\ell_1; \ell_3 := 1 + !\ell_3; \text{unlock } m)$$

► fine-grained:

$$e'_{fg} = (\text{lock } m_1; \ell_1 := 1 + !\ell_1; \text{unlock } m_1; \text{lock } m_2; \ell_2 := 1 + !\ell_2; \text{unlock } m_2) \\ \mid (\text{lock } m_1; \ell_1 := 1 + !\ell_1; \text{unlock } m_1; \text{lock } m_3; \ell_3 := 1 + !\ell_3; \text{unlock } m_3)$$

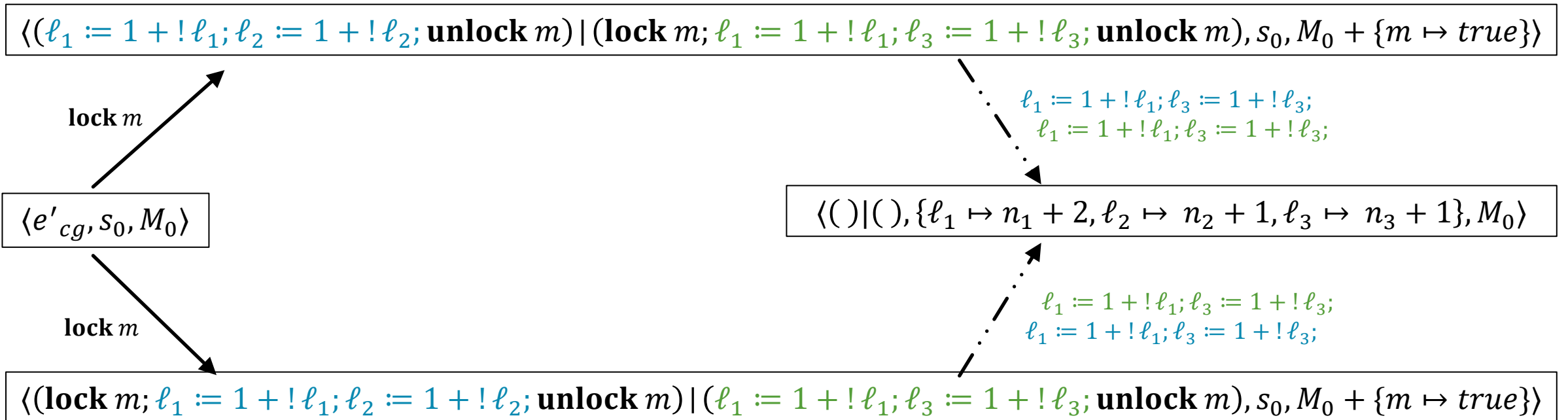
Esecuzione coarse-grained

$$e'_{cg} = (\text{lock } m; \ell_1 := 1 + !\ell_1; \ell_2 := 1 + !\ell_2; \text{unlock } m) \\ | (\text{lock } m; \ell_1 := 1 + !\ell_1; \ell_3 := 1 + !\ell_3; \text{unlock } m)$$

$$s_0 = \{\ell_1 \mapsto n_1, \ell_2 \mapsto n_2, \ell_3 \mapsto n_3\}$$

$$M_0 = \{\forall m. (m \mapsto \text{false})\}$$

Sistema di transizioni (possibili esecuzioni):



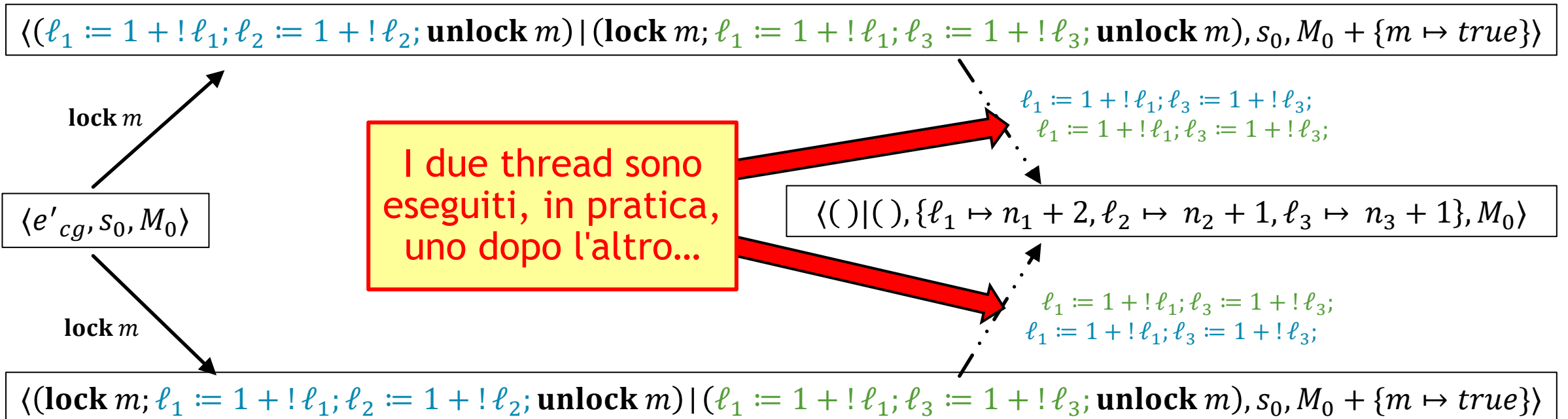
Esecuzione coarse-grained

$$e'_{cg} = (\text{lock } m; \ell_1 := 1 + !\ell_1; \ell_2 := 1 + !\ell_2; \text{unlock } m) \\ | (\text{lock } m; \ell_1 := 1 + !\ell_1; \ell_3 := 1 + !\ell_3; \text{unlock } m)$$

$$s_0 = \{\ell_1 \mapsto n_1, \ell_2 \mapsto n_2, \ell_3 \mapsto n_3\}$$

$$M_0 = \{\forall m. (m \mapsto \text{false})\}$$

Sistema di transizioni (possibili esecuzioni):



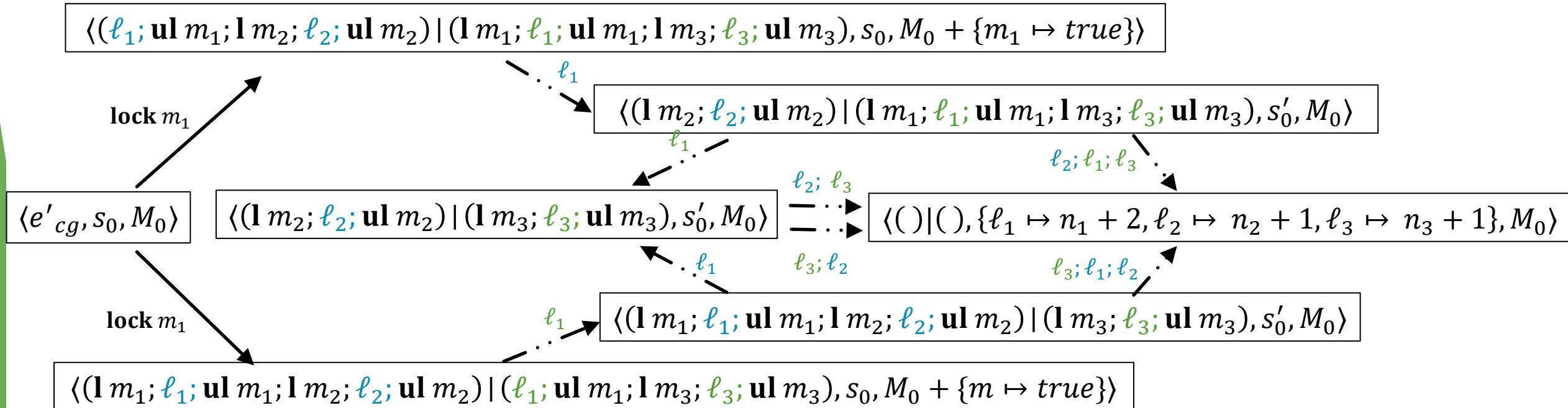
Esecuzione fine-grained

$$s_0 = \{\ell_1 \mapsto n_1, \ell_2 \mapsto n_2, \ell_3 \mapsto n_3\}$$

$$M_0 = \{\forall m. (m \mapsto \mathbf{false})\}$$

$$e'_{fg} = (\mathbf{lock } m_1; \ell_1 := 1 + !\ell_1; \mathbf{unlock } m_1; \mathbf{lock } m_2; \ell_2 := 1 + !\ell_2; \mathbf{unlock } m_2) \\ | (\mathbf{lock } m_1; \ell_1 := 1 + !\ell_1; \mathbf{unlock } m_1; \mathbf{lock } m_3; \ell_3 := 1 + !\ell_3; \mathbf{unlock } m_3)$$

Sistema di transizioni (possibili esecuzioni):



Esecuzione fine-grained

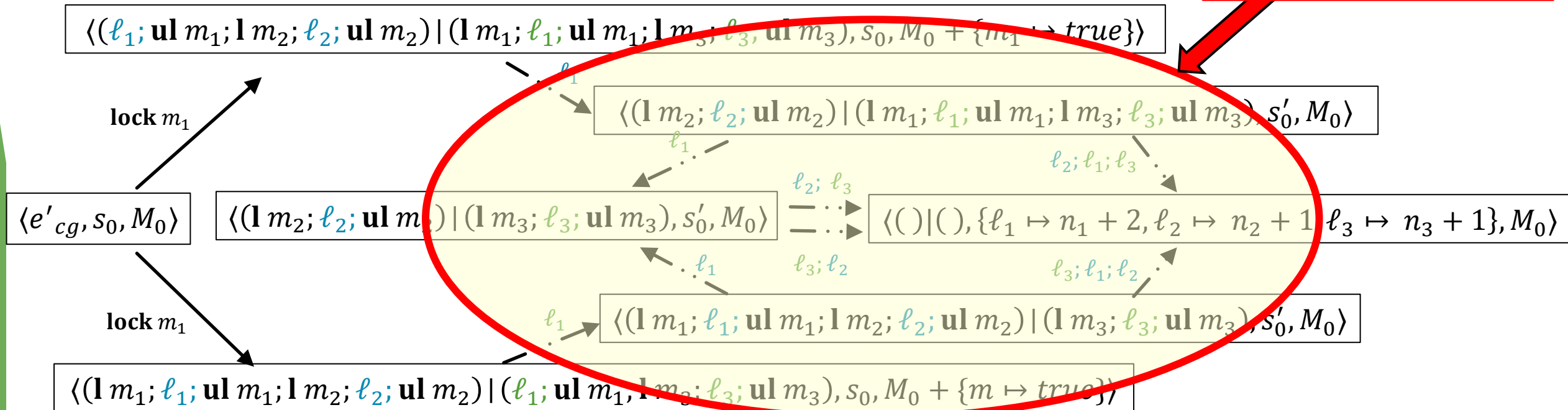
$$s_0 = \{\ell_1 \mapsto n_1, \ell_2 \mapsto n_2, \ell_3 \mapsto n_3\}$$

$$M_0 = \{\forall m. (m \mapsto \text{false})\}$$

$$e'_{fg} = (\text{lock } m_1; \ell_1 := 1 + !\ell_1; \text{unlock } m_1; \text{lock } m_2; \ell_2 := 1 + !\ell_2; \text{unlock } m_2) \\ | (\text{lock } m_1; \ell_1 := 1 + !\ell_1; \text{unlock } m_1; \text{lock } m_3; \ell_3 := 1 + !\ell_3; \text{unlock } m_3)$$

Maggiore libertà
per l'esecuzione
concorrente (o
parallela)

Sistema di transizioni (possibili esecuzioni):



Uso pericoloso dei lock...

Le strategie **fine-grained** sono quindi spesso **preferibili**, ma presentano un altro **problema**...

$$e = (\text{lock } m_1; \text{lock } m_2; \ell_1 := !\ell_2; \text{unlock } m_1; \text{unlock } m_2) \\ | (\text{lock } m_2; \text{lock } m_1; \ell_2 := !\ell_1; \text{unlock } m_1; \text{unlock } m_2)$$

Che comportamento ha questo programma?

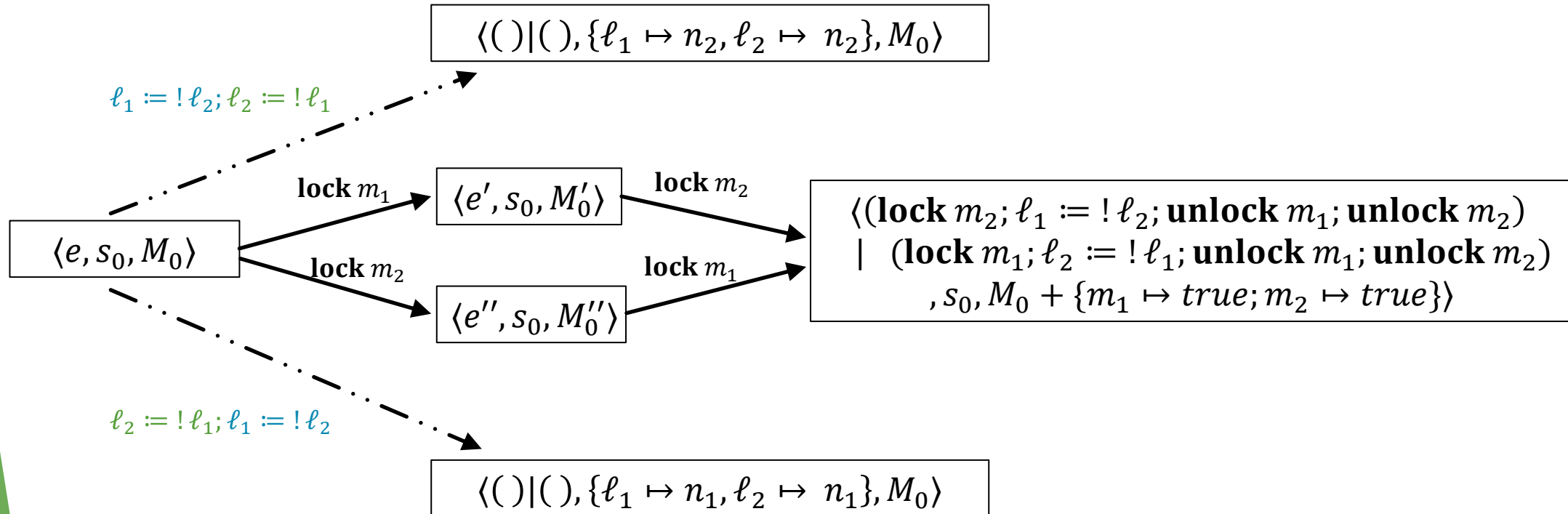
Uso pericoloso dei lock...

$e = (\text{lock } m_1; \text{lock } m_2; \ell_1 := !\ell_2; \text{unlock } m_1; \text{unlock } m_2)$
 $\mid (\text{lock } m_2; \text{lock } m_1; \ell_2 := !\ell_1; \text{unlock } m_1; \text{unlock } m_2)$

$s_0 = \{\ell_1 \mapsto n_1, \ell_2 \mapsto n_2\}$

$M_0 = \{\forall m. (m \mapsto \text{false})\}$

Sistema di transizioni (possibili esecuzioni):



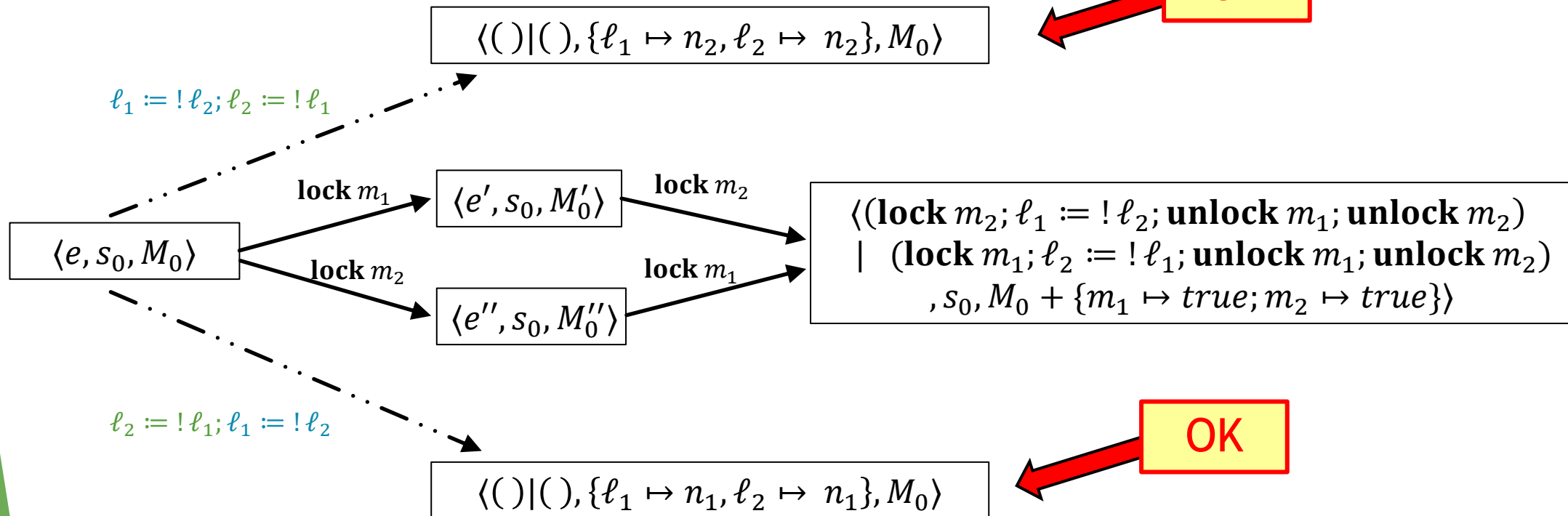
Uso pericoloso dei lock...

$e = (\text{lock } m_1; \text{lock } m_2; \ell_1 := !\ell_2; \text{unlock } m_1; \text{unlock } m_2)$
 $\mid (\text{lock } m_2; \text{lock } m_1; \ell_2 := !\ell_1; \text{unlock } m_1; \text{unlock } m_2)$

$s_0 = \{\ell_1 \mapsto n_1, \ell_2 \mapsto n_2\}$

$M_0 = \{\forall m. (m \mapsto \text{false})\}$

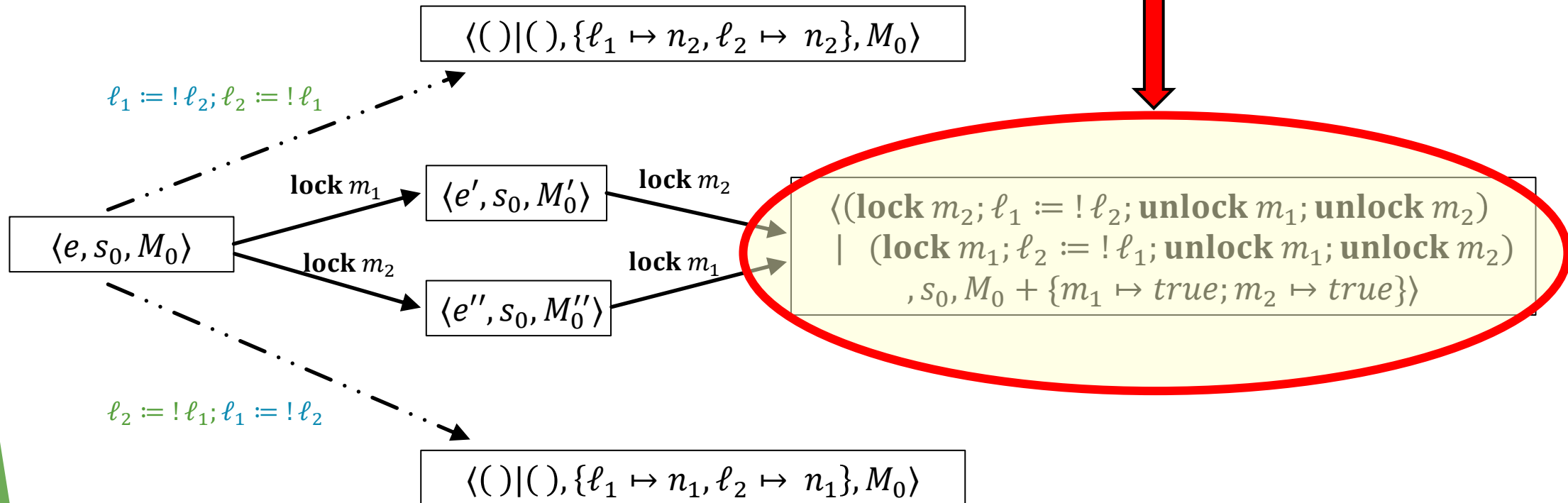
Sistema di transizioni (possibili esecuzioni):



Uso pericoloso dei lock...

$e = (\text{lock } m_1; \text{lock } m_2; \ell_1 := !\ell_2; \text{unlock } m_1; \text{unlock } m_2)$
| $(\text{lock } m_2; \text{lock } m_1; \ell_2 := !\ell_1; \text{unlock } m_1; \text{unlock } m_2)$

Sistema di transizioni (possibili esecuzioni):



Il programma si BLOCCA:

Ogni thread ha acquisito un mutex, ma è bloccato nell'attesa che si liberi l'altro (**attesa circolare**)

Deadlock

La situazione in cui il programma si blocca (definitivamente) a causa dell'uso improprio dei lock prende il nome di

DEADLOCK

- ▶ Si verifica quando si crea una situazione di attesa circolare, come quella vista nell'esempio precedente (A aspetta B, mentre contemporaneamente B aspetta A, condizione che impedisce a entrambi i thread di progredire)
- ▶ È uno dei motivi per cui a volte le applicazioni in esecuzione sul computer si piantano e devono essere "killate» (interrotte manualmente) per ripristinare il funzionamento del sistema

Come risolvere i problemi di deadlock

Tre strategie principali per affrontare il problema dei deadlock:

- ▶ **Deadlock prevention:** si stabiliscono regole, controllabili staticamente (dal compilatore) sull'uso dei lock che garantiscano che i deadlock non possano verificarsi
- ▶ **Deadlock avoidance:** l'esecuzione del programma è monitorata dal supporto a runtime del linguaggio, che si accorge di quando il programma sta per andare in deadlock e interviene cambiando l'ordine di esecuzione dei thread per evitarlo
- ▶ **Deadlock recovery:** si permette al programma di entrare in deadlock, ma se ciò accade il runtime del linguaggio se ne accorge e interviene successivamente, per ripristinare uno stato senza deadlock

Esempio di deadlock prevention: 2-phase locking (2PL)

Una **disciplina** sull'uso dei lock che **previene** i deadlock è il **2-phase locking (2PL)** basandosi su un loro ordinamento predefinito

Assume che esista un ordinamento dei mutex: $m_1, m_2, \dots$

Stabilisce che un thread che voglia acquisire e poi rilasciare un certo numero di mutex debba fare:

- ▶ i rispettivi **lock** in **ordine crescente** e una volta terminato
- ▶ i rispettivi **unlock** in ordine inverso, ovvero in **ordine decrescente**

Le **sequenze** di lock e unlock devono essere **eseguite completamente**

- ▶ non si può acquisire n mutex, rilasciarne solo una parte e acquisirne altri, neanche se si rispetta l'ordine

Esempio di deadlock prevention: 2-phase locking (2PL)

Assumiamo che esista un'associazione 1:1 tra locazioni e mutex

► associamo ℓ_1 a m_1 , ℓ_2 a m_2 , ecc...

Assumiamo l'ordinamento $m_1, m_2, m_3, \dots$

Come va modificato questo esempio che portava in deadlock?

$$e = (\text{lock } m_1; \text{lock } m_2; \ell_1 := !\ell_2; \text{unlock } m_1; \text{unlock } m_2) \\ | (\text{lock } m_2; \text{lock } m_1; \ell_2 := !\ell_1; \text{unlock } m_1; \text{unlock } m_2)$$

Esempio di deadlock prevention: 2-phase locking (2PL)

$e = (\text{lock } m_1; \text{lock } m_2; \ell_1 := !\ell_2; \text{unlock } m_1; \text{unlock } m_2)$
| $(\text{lock } m_2; \text{lock } m_1; \ell_2 := !\ell_1; \text{unlock } m_1; \text{unlock } m_2)$

NO

$e' = (\text{lock } m_1; \text{lock } m_2; \ell_1 := !\ell_2; \text{unlock } m_2; \text{unlock } m_1)$
| $(\text{lock } m_1; \text{lock } m_2; \ell_2 := !\ell_1; \text{unlock } m_2; \text{unlock } m_1)$

SI

Ordine
crescente

Ordine
decrescente

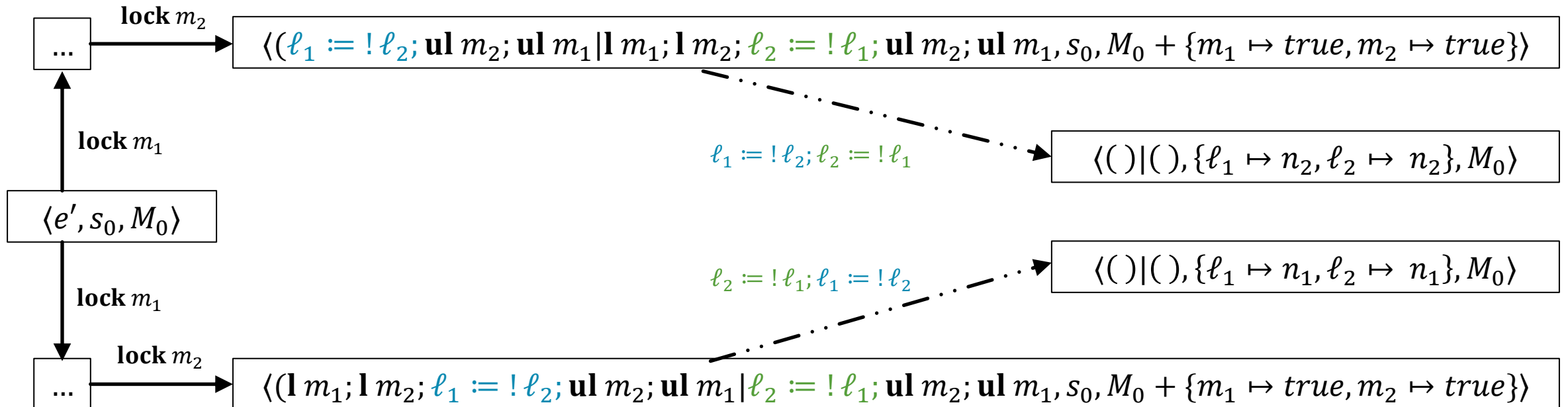
Esempio di deadlock prevention: 2-phase locking (2PL)

$$s_0 = \{\ell_1 \mapsto n_1, \ell_2 \mapsto n_2, \ell_3 \mapsto n_3\}$$

$$M_0 = \{\forall m. (m \mapsto \mathbf{false})\}$$

$$e' = (\mathbf{lock } m_1; \mathbf{lock } m_2; \ell_1 := !\ell_2; \mathbf{unlock } m_2; \mathbf{unlock } m_1) \\ | (\mathbf{lock } m_1; \mathbf{lock } m_2; \ell_2 := !\ell_1; \mathbf{unlock } m_2; \mathbf{unlock } m_1)$$

Sistema di transizioni (possibili esecuzioni):



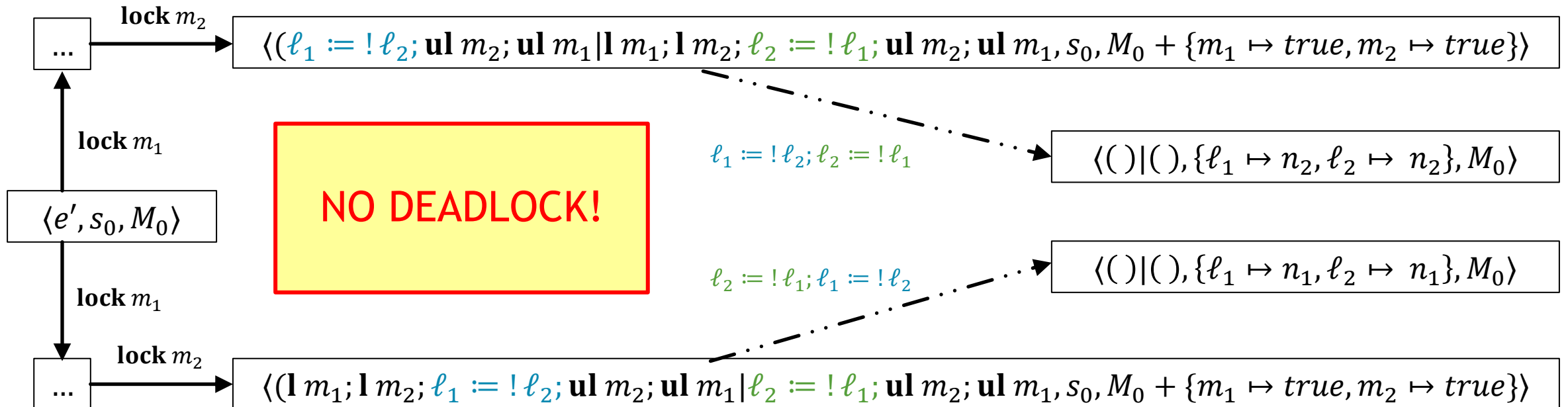
Esempio di deadlock prevention: 2-phase locking (2PL)

$$s_0 = \{\ell_1 \mapsto n_1, \ell_2 \mapsto n_2, \ell_3 \mapsto n_3\}$$

$$M_0 = \{\forall m. (m \mapsto \mathbf{false})\}$$

$$e' = (\mathbf{lock } m_1; \mathbf{lock } m_2; \ell_1 := !\ell_2; \mathbf{unlock } m_2; \mathbf{unlock } m_1) \\ | (\mathbf{lock } m_1; \mathbf{lock } m_2; \ell_2 := !\ell_1; \mathbf{unlock } m_2; \mathbf{unlock } m_1)$$

Sistema di transizioni (possibili esecuzioni):



Riassumendo

Abbiamo definito un **modello di programmazione concorrente** che ci ha consentito di **ragionare** su alcuni **concetti essenziali**:

- ▶ esecuzione non sequenziale
- ▶ mutua esclusione e sincronizzazione
- ▶ deadlock

Molti linguaggi di programmazione offrono **primitive di concorrenza e sincronizzazione di alto livello**, anche molto diverse tra loro

- ▶ aver visto i concetti su un modello semplice sarà di aiuto nella comprensione dei costrutti di alto livello